SOFT-OUTPUT VITERBI ALGORITHM

Dokumentation zum Masterprojekt

angefertigt von

Parivash Abdollah Zadeh, Hamed Saber

bei

Univ.-Prof.Dr.-Ing.habil. Peter Jung

Lehrstuhl für Kommunikationstechnik
Fakultät für Ingenieurwissenschaften
Universität Duisburg-Essen

Duisburg, den 27.01.2025

Inhaltsverzeichnis

Danksagung **2**

Abstract **3**

Übersicht **4**

Arbeitsteilung **5**

1 Soft Output Viterbi Algorithm (SOVA) **6**

1.1 Voraussetzungen *6*

1.1.1 Diskrete Zeit Übertragung 6

1.1.2 Datensymbole und Vektor 6

1.1.3 Antwortkanal 6

1.1.4 Empfangener Vektor 6

1.2 Wahrscheinlichkeitsfunktion *7*

1.2.1 Ungeboeck und Forney metrics 7

1.3 Log-likelihood ratio (LLR) *7*

1.3.1 Simple Rule 7

1.3.2 Huber Rule 7

1.3.3 Battail Rule 8

1.4 Aufgaben *9*

1.5 Referenzen…….…… 30

Danksagung

An dieser Stelle möchten wir unsere aufrichtige Dankbarkeit gegenüber allen Personen ausdrücken, die uns während unseres Masterprojekts unterstützt haben.

unser besonderer Dank gilt unserem Betreuer, Herr M.Sc. Faris Abdel Rehim, der uns während des gesamten Projekts mit seiner Expertise, Geduld und wertvollen Anregungen begleitet hat. Seine Unterstützung und konstruktive Kritik haben maßgeblich zur Qualität dieser Arbeit beigetragen.

Ein großer Dank gilt auch Prof. Dr.-Ing. habil. Peter Jung , der uns die notwendigen Ressourcen, Infrastruktur und Unterstützung zur Verfügung gestellt haben, um unsere Forschung erfolgreich durchzuführen.

Diese Arbeit wäre ohne die Unterstützung all dieser Menschen nicht möglich gewesen. Ihnen allen gilt unser herzlicher Dank.

Duisburg, 27.01.2025
Parivash Abdollah Zadeh, Hamed Saber

Abstract

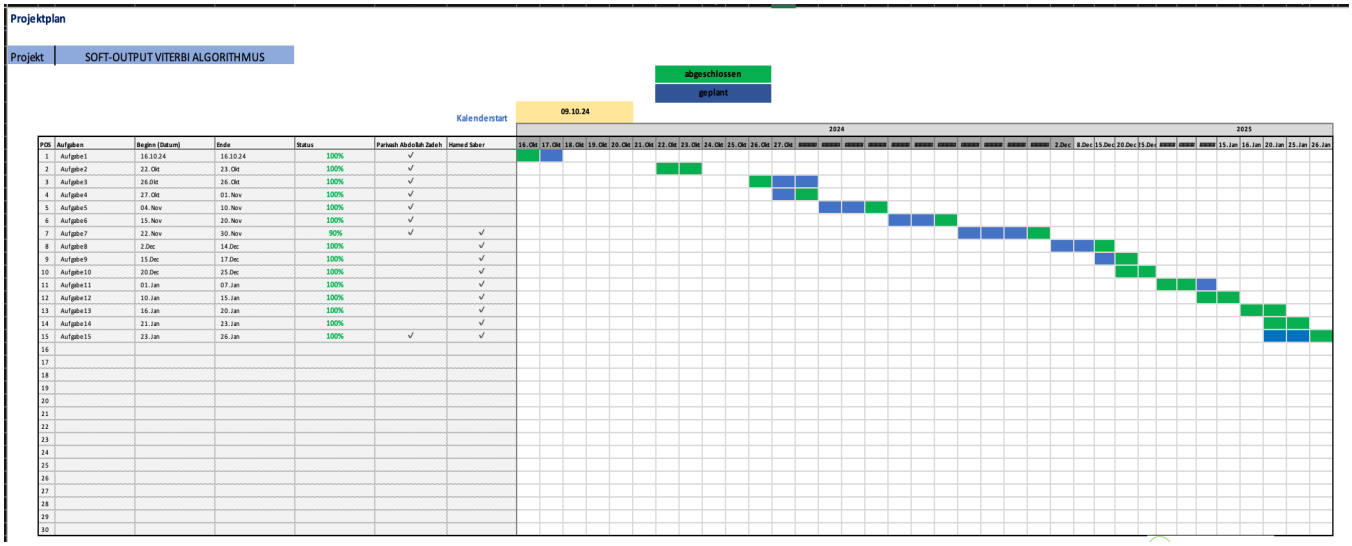
This Master's project focuses on the analysis and optimization of wireless information systems using MATLAB. The objective is to implement and evaluate the Soft Output Viterbi Algorithm (SOVA), a crucial method for enhancing error correction in digital communication systems.
The project starts by modeling a digital transmission channel characterized by a discrete impulse response and white noise.
Both the original binary data symbols and the noisy signals transmitted through the channel are analyzed. A burst model is used to transmit data in segments with defined initial and terminal conditions. A key focus of the work is the development of Log-Likelihood Ratio (LLR) computation rules.
The Simple Rule, Huber Rule, and Battail Rule are implemented to evaluate uncertainties in bit decisions. This facilitates more accurate error correction, especially in noisy channel conditions. Additionally, a MATLAB script is developed to support the analysis and visualization of channel dynamics.
The script generates trellis diagrams to represent state transitions and highlights differences between survivor and competing paths. Transmission errors are localized, and LLR values are visualized for different rules.
Finally, the system's performance is evaluated under varying signal-to-noise ratio (SNR) conditions. The results demonstrate that selecting appropriate LLR rules and optimizing detection algorithms are critical for reliable data transmission.

Übersicht

Dieses Masterprojekt beschäftigt sich mit der Analyse und Optimierung drahtloser Informationssysteme mithilfe von MATLAB. Im Zentrum steht die Implementierung und Untersuchung des Soft Output Viterbi Algorithmus (SOVA), einer Methode, die entscheidend zur Verbesserung der Fehlerkorrektur und Zuverlässigkeit moderner Kommunikationssysteme beiträgt.
Zunächst wird ein digitales Übertragungssysteme modelliert, das verrauschte Kanäle simuliert, die häufig in realen drahtlosen Kommunikationssystemen auftreten. Die Modellierung umfasst die Definition eines diskreten Zeitkanals, der durch seine Impulsantwort und die Hinzufügung von weißem Rauschen charakterisiert wird. Dieses Kanalmodell wird genutzt, um die Übertragung und Detektion binärer Daten zu simulieren und dabei sowohl die ursprünglichen als auch die empfangenen Signale zu analysieren. Ein zentraler Bestandteil der Arbeit ist die Implementierung von Log-Likelihood-Ratio (LLR) Berechnungsregeln, darunter die Simple Rule, die Huber Rule und die Battail Rule.
Diese Regeln ermöglichen es, Un- sicherheiten bei der Bit-Entscheidung zu bewerten und so die Fehlerkorrektur durch SOVA zu optimieren. Im Projekt wird gezeigt, wie sich die Wahl der LLR-Regel auf die Genauigkeit der Datenrekonstruktion auswirkt. Darüber hinaus wird ein umfassendes MATLAB-Skript entwickelt, das die Visualisierung der Kanal- und Detektionsprozesse unterstützt. Das Skript erstellt Trellis-Diagramme, die die Zustandsübergänge im Kanal darstellen, und hebt die Unterschiede zwischen dem idealen, dem detektierten und den konkurrierenden Pfaden hervor.
Zusätzlich werden Übertragungsfehler und deren Lokalisierung visualisiert. Abschließend

wird die Leistung des Systems in Szenarien mit unterschiedlichen Signal-Rausch-Verhältnissen (SNR) untersucht. Die Ergebnisse zeigen, dass die Wahl geeigneter LLR-Regeln und die Optimierung der Detektionsalgorithmen entscheidend für die Verbesserung der Übertragungsqualität sind. Dieses Projekt bietet einen wertvollen Beitrag zur Entwicklung robuster Kommunikationssystem.

Arbeitsteilung



Projektplan

Projekt	SOFT-OUTPUT VITERBI ALGORITHMUS

abgeschlossen
geplant

Kalenderstart: 09.10.24

POS	Aufgaben	Beginn (Datum)	Ende	Status	Parwish Abdollah Zadeh	Hamed Saber
1	Aufgabe1	16.10.24	16.10.24	100%	✓	
2	Aufgabe2	22. Okt	23. Okt	100%	✓	
3	Aufgabe3	26.Okt	26. Okt	100%	✓	
4	Aufgabe4	27. Okt	01. Nov	100%	✓	
5	Aufgabe5	04. Nov	10. Nov	100%	✓	
6	Aufgabe6	15. Nov	20. Nov	100%	✓	
7	Aufgabe7	22. Nov	30. Nov	90%	✓	✓
8	Aufgabe8	2.Dec	14.Dec	100%		✓
9	Aufgabe9	15.Dec	17.Dec	100%		✓
10	Aufgabe10	20.Dec	25.Dec	100%		✓
11	Aufgabe11	01. Jan	07. Jan	100%		✓
12	Aufgabe12	10. Jan	15. Jan	100%		✓
13	Aufgabe13	16. Jan	20. Jan	100%		✓
14	Aufgabe14	21. Jan	23. Jan	100%		✓
15	Aufgabe15	23. Jan	26. Jan	100%	✓	✓

Soft Output Viterbi Algorithm (SOVA)

1.1. Voraussetzungen

1.1.1 Diskrete Zeitpunkt:

Diskrete Zeitübertragung beschreibt die Übertragung und Verarbeitung von Signalen, die in festen Zeitintervallen abgetastet werden. Dies ist typisch für digitale Systeme wie Steuerungen, Signalverarbeitung oder digitale Kommunikation.

1.1.2 Datensymbole und Vektor:

Diese Daten repräsentieren die einzelnen empfangenen oder kodierten Werte in einem Kommunikationssystem, oft als diskrete Symbole in einem Modulationsschema Vektoren:
In jedem Schritt des Viterbi-Algorithmus werden Zustände durch Wahrscheinlichkeitswerte und Pfadinformationen als Vektoren dargestellt. Diese Vektoren enthalten:

1. Zustandswahrscheinlichkeiten: Wahrscheinlichkeiten für jeden möglichen Zustand.
2. Pfadinformationen: Informationen über die vorhergehenden Zustände zur Wiederherstellung der optimalen Sequenz.

1.1.3 Antwortkanal:

Der Antwortkanal (Response Channel) ist der Kanal, durch den die kodierten Daten in einem Kommunikationssystem übertragen werden.

$$\boldsymbol{h} = \left(h_1, h_2 \cdots h_W\right)^{\mathrm{T}}, \quad h_w \in \mathbb{R}, \quad w = 1,2 \cdots W, \quad W \in \mathbb{N}^*$$

1.1.4 Empfangener Vektor:

Der empfangene Vektor besteht aus den Werten, die nach der Übertragung über den Kanal ankommen. Dieser Vektor enthält oft verrauschte und möglicherweise verzerrte Versionen der ursprünglich gesendeten Daten.

$$e_n = \sum_{w=1}^{W} h_w d_{n-w+1} + n_n, \quad e_n \in \mathbb{R}.$$

1.2 Wahrscheinlichkeitsfunktion :

1.2.1 Ungerböck und Forney Metriks:

Das ML-Erkennungsprinzip wird basierend auf dem Kanalrauschmodell definiert. Unter der Annahme, dass das Kanalrauschen eine Gauß-Verteilung mit der Varianz $\boldsymbol{\delta^2}$ hat,

gilt:

$$p(e|d) \propto \exp\left(-\frac{1}{2\sigma^2}\sum_{N+W}^{N+2(W-1)}\left(e_n - \sum_{w=1}^{W} h_w b_{n-w+1}\right)^2\right)$$

Um die obige Wahrscheinlichkeit zu maximieren, muss der folgende Ausdruck minimiert werden:

$$\hat{d} = \underset{d}{\operatorname{argmin}} \sum_{n=W}^{N+2(W-1)}\left(e_n - \sum_{w=1}^{W} h_w b_{n-w+1}\right)^2$$

Zur Optimierung dieses Problems kann der Viterbi-Algorithmus verwendet werden .Indem diese Beziehung maximiert wird, können die Werte für die Forney- und Ungerboeck-Metriken berechnet werden.

1.3 Log Likelihood-Ratio(LLR) :

1.3.1 SIMPLE RULE :

Diese Regel berechnet den LLR-Wert für jeden Zustand im Zeitpunkt nn wie folgt:

$$L_{\mathrm{s},n} = \Delta_n^{(\mathrm{s})}, \quad n = 1,2\cdots N\,,$$

Dabei ist $\Delta_n^{(s)}$ die Differenz oder ein spezifischer Wert, der mit dem Zustand **s** im Zeitpunkt **n** assoziiert ist.

1.3.2 Die Huber-Regel (HUBER RULE):

Bei dieser Regel werden die LLR-Werte rekursiv aktualisiert und basieren auf verschiedenen Bedingungen:

Diese Beziehung zeigt, dass die LLR-Werte entlang des Überlebenspfads abhängig von der Übereinstimmung der Daten mit dem Kandidatenpfad und den vorherigen Zuständen aktualisiert werden.

$$L_{\mathrm{h},\kappa}^{(\mathrm{s}),\mathrm{new}} = \begin{cases} \Delta_n^{(\mathrm{s})} & \kappa = n, \\ L_{\mathrm{h},\kappa}^{(\mathrm{s})} & \kappa < n,\ \underline{d}_\kappa^{(\mathrm{s})} = \underline{d}_\kappa^{(\mathrm{c})}, \\ \min\left\{L_{\mathrm{h},\kappa}^{(\mathrm{s})}, \Delta_n^{(\mathrm{s})}\right\} & \kappa < n,\ \underline{d}_\kappa^{(\mathrm{s})} \neq \underline{d}_\kappa^{(\mathrm{c})}, \end{cases}$$

1.3.3 Die Battail-Regel (BATTAIL RULE):

Diese Regel aktualisiert die LLR-Werte ebenfalls rekursiv und verwendet eine komplexere Kombination:

$$L_{\mathrm{b},\kappa}^{(\mathrm{s}),\mathrm{new}} = \begin{cases} \Delta_n^{(\mathrm{s})} & \kappa = n, \\ \min\left\{ L_{\mathrm{b},\kappa}^{(\mathrm{s})}, \left[\Delta_n^{(\mathrm{s})} + L_{\mathrm{b},\kappa}^{(\mathrm{c})} \right] \right\} & \kappa < n,\, \underline{d}_\kappa^{(\mathrm{s})} = \underline{d}_\kappa^{(\mathrm{c})}, \\ \min\left\{ L_{\mathrm{b},\kappa}^{(\mathrm{s})}, \Delta_n^{(\mathrm{s})} \right\} & \kappa < n,\, \underline{d}_\kappa^{(\mathrm{s})} \neq \underline{d}_\kappa^{(\mathrm{c})}. \end{cases}$$

1.4 Die Aufgaben

1.4.1. Aufgabe 1 :

Determine the burst vector $\boldsymbol{b}$ which consists of the coordinates $b_n \in \{-1, +1\}$!

Es müssen **W−1** Bits am Anfang und **W−1** Bits am Ende der gesendeten Sequenz hinzugefügt werden. Wie in der Aufgabenstellung angegeben, sind all diese Bits auf **-1** gesetzt. Die Länge der ursprünglichen gesendeten Sequenz ohne die zusätzlichen Bits beträgt **N** und wird wie folgt dargestellt:

$$d = (d_1, d_2, \dots, d_N)^T$$

Durch das Hinzufügen von Bits am Anfang und Ende, um den Burst-Vektor **b** zu erstellen, ergibt sich die endgültige Länge des Vektors als:

$$N + 2(W - 1)$$

Daher ergibt sich der Burst-Vektor bb wie folgt:

$$b = \left[\underbrace{-1, -1, \dots, -1}_{W-1}, d_1, d_2, \dots, d_N, \underbrace{-1, -1, \dots, -1}_{W-1}\right]^T$$

Somit ist der endgültige Burst-Vektor, der die Werte -1 und 1 enthält, wie oben dargestellt.

1.4.2. Aufgabe 2:

How many samples do the noise vector $\boldsymbol{n}$ and the received vector $\boldsymbol{e}$ contain?

Die Kanalantwort hat ***W*** Abtastwerte. Dies bedeutet, dass die Systemantwort auf die gesendeten Daten über **W** Zeitschritte verteilt ist. Am Empfänger wird das Signal mit der Kanalantwort gefaltet. Das bedeutet, dass jedes gesendete Symbol **W** Abtastwerte des empfangenen Signals beeinflusst.

Daher beträgt die Gesamtanzahl der empfangenen Abtastwerte für **N** gesendete Symbole ohne Burst:

$$N + 2(W - 1) + W - 1 = N + 3W - 3 = N + 3(W - 1)$$

Entsprechend enthalten die Vektoren ***n*** und ***e*** jeweils **N+3(W−1)** Abtastwerte.

1.4.3. Aufgabe 3:

Determine the noise vector $\boldsymbol{n}$ and the received vector $\boldsymbol{e}$ in explicit form following the example of (7.54)!

Basierend auf den Überlegungen zu Frage 1 und Frage 2 ergibt sich:

$$e = b * h + n$$

oder in expliziter Form:

$$e_n = \sum_{w=1}^{W} h_w b_{n-w+1} + n_n$$

Da der Rauschvektor ein weißes Rauschen darstellt, sind die einzelnen Abtastwerte unabhängig, haben einen Mittelwert von **Null**[1] und eine Leistungsdichte des Rauschens von $\boldsymbol{N_0}$. Daher hat der Rauschvektor **N + 3(W-1)** Abtastwerte.

1.4.4. Aufgabe 4 :

Considering the Mealy model, determine the state transition function δ and the output function ω of the finite state machine which forms the discrete-time transmission channel according to Fig. 7.11!

Der Zustandsdiagramm des Mealy-Modells enthält die Schnittpunkte der Ausgangswerte mit jeder Übergangskante. Eine einfache Mealy-Maschine hat eine Eingabe und eine Ausgabe. Jede Übergangskante ist mit einem Eingabewert und einem Ausgabewert beschriftet. Der Startzustand des Modells ist $\boldsymbol{S_i}$.

In dem im Beispiel beschriebenen Modell beträgt die Kanalimpulsantwort **W=5**. Daher hat das System **W−1= 4** Speicherkomponenten. Dieser Speicher verursacht Inter-Symbol-Interferenz (ISI). Der Zustand zum Zeitpunkt **n** umfasst die **W−1** vorherigen gesendeten Bits :

$$S_n = \left(b_{n-(w-1)}, \dots, b_n\right)$$ [2]

Da **W=4** ist, gilt:

$$S_n = (b_{n-3}, b_{n-2}, b_{n-1}, b_n)$$

Somit gibt es **2^4 = 16** mögliche Zustände.

---

[1]

[2] Prof.Peter Jung : Advanced Mobile Communications

Die Zustandsübergangsfunktion **δ** zeigt, wie das Modell vom Zustand $\boldsymbol{S_n}$ in den Zustand $\boldsymbol{S_{n+1}}$ übergeht, wenn $\boldsymbol{b_n}$ gesendet wird. Der neue Zustand lautet:

$$S_{n+1} = \left(b_{n-(w-2)}, \dots, b_{n+1}\right)$$ [3]

Angenommen, **W=4** , dann gilt:

$$S_{n+1} = (b_{n-2}, b_{n-1}, b_n, b_{n+1})$$

Die Ausgabe des Zustandsmodells **ω** ist die Kanalausgabe $\boldsymbol{e_n}$. Diese ist die Faltung der Eingangsbits mit der Kanalantwort, ergänzt durch Rauschen.

Für die Zustandsübergangsfunktion können wir schreiben :

$$\delta(S_n, b_{n+1}) = (b_{n-3}, b_{n-2}, b_{n-1}, b_n, b_{n+1})$$

Die Ausgabe $\boldsymbol{e_n}$ hängt vom aktuellen Zustand und dem neuen Eingangsbit $\boldsymbol{b_n}$ ab :

$$e_n = \sum_{w=1}^{W} h_w b_{n-w+1}$$

$$e_n = h_1 b_n + h_2 b_{n-1} + h_3 b_{n-2} + h_4 b_{n-3} + h_5 b_{n-4} = \omega(S_n, b_n)$$

Beispiel: Angenommen, $S_n = (1,1,-1,1)$ und $b_n = -1$. Basierend auf der Kanalimpulsantwort:

$$\delta(S_n, b_{n+1}) = \ S_{n+1} = (-1,1,1,-1)$$

Die Ausgabe lautet:

$$e_n = 1 * -1 + \frac{1}{2} * 1 + \frac{1}{4} * 1 + \frac{1}{2} * -1 + \frac{1}{4} * 1 = -\frac{1}{2} = \omega(S_n, b_n)$$

1.4.5. Aufgabe 5:

[3] Prof.Peter Jung : Advanced Mobile Communications

Let us now consider that we would like to detect the data sequence $\boldsymbol{d}$ of (7.54) using an ML (maximum likelihood) detector. We denote the detected data sequence by

$$\hat{\boldsymbol{d}} = \left(\hat{d}_1, \hat{d}_2 \cdots \hat{d}_N\right)^{\mathrm{T}}, \quad \hat{d}_n \in \{-1, +1\}, n = 1, 2 \cdots N, N \in \mathbb{N}^*. \tag{7.67}$$

The ML detection rule is given by

$$\hat{\boldsymbol{d}} = \arg\max_{\boldsymbol{d}} \left\{ p\left(\boldsymbol{e} | \boldsymbol{d}\right) \right\}. \tag{7.68}$$

Bear in mind that we will transmit the data in bursts which commence with $W-1$ binary tailbits and end in $W-1$ binary tailbits, all tailbits being equal to -1. The tailbits are known to both the transmitter and the receiver.

Prior to the completion of the transmission of the $W-1$ initial tailbits, the state of the discrete-time transmission channel is undetermined. Therefore, we cannot give any reasonable mathematical determination in the case of $n < W$. We will therefore focus on $n \geq W$.

Furthermore, owing to the length of the burst vector $\boldsymbol{b}$, $N + 2\left(W-1\right)$, n must not be greater than $N + 2\left(W-1\right)$.

Clearly, the initial state of discrete-time multipath channel is

$$\boldsymbol{s}_{W-1} = \left(\underbrace{-1, -1 \cdots -1}_{W-1 \text{ tailbits}} \right)^{\mathrm{T}} \tag{7.69}$$

with the state number

$$s_{W-1} = \sum_{w=0}^{W-2} \frac{-1+1}{2} 2^w = \sum_{w=0}^{W-2} 0 = 0, \tag{7.70}$$

and the terminal state is the state

$$\boldsymbol{s}_{N+2(W-1)} = \left(\underbrace{-1, -1 \cdots -1}_{W-1 \text{ tailbits}} \right)^{\mathrm{T}}. \tag{7.71}$$

with the state number

$$s_{N+2(W-1)} = \sum_{w=0}^{W-2} \frac{-1+1}{2} 2^w = \sum_{w=0}^{W-2} 0 = 0. \tag{7.72}$$

Determine the ML detector!

Gemäß der Definition des ML-Detektors (**Maximum Likelihood**) wird die gesendete Datenfolge so bestimmt, dass die Wahrscheinlichkeit **p(e|d)** maximiert wird.

$$\hat{d} = (d_1, d_2, \dots, d_N)^T$$

Wenn wir dies mathematisch darstellen möchten, basiert das Entscheidungsmaß für diese Erkennung auf der folgenden Gleichung:

$$\hat{d} = \underset{d}{\operatorname{argmax}}\, p(e|d)$$

Unter Berücksichtigung der diskreten Kanalimpulsantwort und des Kanalrauschens kann der Vektor des empfangenen Signals wie folgt dargestellt werden :

$$e_n = \sum_{w=1}^{W} h_w b_{n-w+1} + n_n$$

Der Vektor ***b*** in der obigen Gleichung ist der Burst-Vektor. Die Werte dieses Vektors wurden in der Antwort zur ersten Frage angegeben.

Wir wissen, dass der Kanalzustand zu jedem Zeitpunkt ***n*** durch die vorherigen **W−1** Bits bestimmt wird. Daher kann man schreiben:

$$S_n = (b_{n-(w-1)}, \dots, b_n)$$

Basierend auf der Struktur des Burst-Vektors werden der Anfangszustand und der Endzustand wie folgt definiert:

$$S_{W-1} = [-1, -1, \dots, -1]^T$$

$$S_{N+2(W-1)} = [-1, -1, \dots, -1]^T$$

Das ML-Erkennungsprinzip wird basierend auf dem Kanalrauschmodell definiert. Unter der Annahme, dass das Kanalrauschen eine Gauß-Verteilung mit der Varianz $\boldsymbol{\delta^2}$ hat,

gilt:

$$p(e|d) \propto \exp\left(-\frac{1}{2\sigma^2}\sum_{N=W}^{N+2(W-1)}\left(e_n - \sum_{w=1}^{W} h_w b_{n-w+1}\right)^2\right)$$

Um die obige Wahrscheinlichkeit zu maximieren, muss der folgende Ausdruck minimiert werden:

$$\hat{d} = \underset{d}{\operatorname{argmin}} \sum_{n=W}^{N+2(W-1)}\left(e_n - \sum_{w=1}^{W} h_w b_{n-w+1}\right)^2$$

Zur Optimierung dieses Problems kann der Viterbi-Algorithmus verwendet werden. Dieser Algorithmus ist besonders für Systeme mit Speicher hervorragend geeignet. Die Zustände im Viterbi-Algorithmus für dieses Problem werden wie folgt definiert:

$$S_n = (b_{n-(w-1)}, \dots, b_n)$$

Dann gibt $\mathbf{2^{W-1}}$ Zustände .

Für den Übergang vom Zustand $\boldsymbol{S_{n-1}}$ zum Zustand $\boldsymbol{S_n}$ bei einem Eingabebit $\boldsymbol{b_n}$ wird eine Metrik definiert, die als Zweigmetrik bezeichnet wird. Diese Metrik kann wie folgt berechnet werden:

$$f_n = (e_n - \sum_{w=1}^{W} h_w b_{n-w+1})^2$$

Die Pfadmetrik im Viterbi-Algorithmus summiert die Zweigmetriken vom Anfangszustand bis zum aktuellen Zustand. Die Pfadmetrik im Zustand $\boldsymbol{S_n}$ lautet daher:

$$F_n(S_n) = \min_{S_{n-1}, b_n} [F_{n-1}(S_{n-1}) + f_n]$$

Die Metrik des anfänglichen Pfads für den anfänglichen Zustand $\boldsymbol{S_{W-1}}$, der oben definiert wurde, beträgt **null** .

$$F_{W-1}(S_{W-1}) = 0$$

Daher muss für jeden Zustand $\boldsymbol{S_n}$ die Pfadmetrik $\boldsymbol{F_n(S_n)}$ unter Berücksichtigung aller möglichen Übergänge von den vorherigen Zuständen $\boldsymbol{S_{n-1}}$ berechnet werden.

Für den End- oder Zielzustand, der oben erwähnt wurde, wird der Pfad mit der minimalen Metrik ausgewählt.

$$\hat{d} = \operatorname{argmin} F_{N+2(W-1)}(S_{N+2(W-1)})$$

Zusammenfassend haben wir:

- Die Zustände werden durch die letzten W-1 Bits definiert.
- Die Zweigmetriken werden anhand des euklidischen Abstands zwischen dem empfangenen Signal und dem angenommenen Kanalausgang berechnet.
- Die Pfadmetriken ergeben sich aus der Summe der Zweigmetriken über die Zeit.
- Die Anfangs- und Endzustände sind festgelegt.

1.4.6. Aufgabe 6:

Setting out from (7.57), determine the Forney metric increment $f_n^{(\text{Forney})}$ and the Ungerboeck metric increment $f_n^{(\text{Ungerboeck})}$!

Die Erhöhung der Forney-Metrik entspricht dem quadrierten euklidischen Abstand zwischen dem empfangenen Signal ($\boldsymbol{e_n}$) und dem geschätzten Signal ($\boldsymbol{\hat{e}_n}$) und wird wie folgt definiert :[4]

$$f_n^{Forney} = (e_n - \hat{e}_n)^2$$

Für einen Kanal mit der gegebenen Impulsantwort haben wir:

$$\hat{e}_n = \sum_{w=1}^{W} h_w b_{n-w+1}$$

Daher haben wir für die Forney-Metrik:

$$f_n^{Forney} = \left(e_n - \sum_{w=1}^{W} h_w b_{n-w+1}\right)^2$$

Die UngerBoeck-Metrik verwendet die Kreuzkorrelation zwischen dem empfangenen Signal und den angenommenen gesendeten Symbolen und wird wie folgt definiert [5]:

$$f_n^{ungerboeck} = -2e_n.\hat{e}_n + \hat{e}_n^2$$

Die obige Beziehung kann durch Einsetzen wie folgt geschrieben werden:

$$f_n^{ungerboeck} = -2e_n.\sum_{w=1}^{W} h_w b_{n-w+1} + \left(\sum_{w=1}^{W} h_w b_{n-w+1}\right)^2$$

Auf dieser Grundlage wurde die Erhöhung für die beiden genannten Metriken bestimmt. Angenommen, die Impulsantwort des Kanals ist wie folgt:

$$h = \left(1, \frac{1}{2}, \frac{1}{4}, \frac{1}{2}, \frac{1}{4}\right)^T$$

Die angenommene gesendete Sequenz ist gleich $\boldsymbol{b_{n-k}}$ für k von **0** bis **4**. Der Wert von $\boldsymbol{\hat{e}_n}$ ist:

$$\hat{e}_n = \sum_{w=1}^{W} h_w b_{n-w+1} = b_n + \frac{1}{2}b_{n-1} + \frac{1}{4}b_{n-2} + \frac{1}{2}b_{n-3} + \frac{1}{4}b_{n-4}$$

---

[4] Forney, G. D. Jr., "The Viterbi Algorithm," *Proceedings of the IEEE*, 1973

[5] G. Ungerboeck, "Channel coding with multilevel/phase signals," in IEEE Transactions on Information Theory, vol. 28, no. 1, pp. 55-67, January 1982, doi: 10.1109/TIT.1982.1056454.

Daher ist die Zunahme der Metrik für die beiden genannten Metriktypen gleich:

$$f_n^{Forney} = \left(e_n - \sum_{w=1}^{W} h_w b_{n-w+1}\right)^2 = \left(e_n - b_n - \frac{1}{2}b_{n-1} - \frac{1}{4}b_{n-2} - \frac{1}{2}b_{n-3} - \frac{1}{4}b_{n-4}\right)^2$$

$$\begin{aligned} f_n^{ungerboeck} &= -2e_n \cdot \sum_{w=1}^{W} h_w b_{n-w+1} + \left(\sum_{w=1}^{W} h_w b_{n-w+1}\right)^2 \\ &= -2e_n \left(b_n + \frac{1}{2}b_{n-1} + \frac{1}{4}b_{n-2} + \frac{1}{2}b_{n-3} + \frac{1}{4}b_{n-4}\right) \\ &+ \left(b_n + \frac{1}{2}b_{n-1} + \frac{1}{4}b_{n-2} + \frac{1}{2}b_{n-3} + \frac{1}{4}b_{n-4}\right)^2 \end{aligned}$$

1.4.7. Aufgabe 7 :

Implement a MATLAB script for the evaluation of the SOVA!

The MATLAB script shall support the variation of the number of discrete-time paths W.

The MATLAB script shall support the use of discrete-time channel impulse response vectors $\|\boldsymbol{h}\|$ which are normalized, i.e. the discrete-time channel impulse response shall all fulfill

$$\|\boldsymbol{h}\| = 1. \tag{7.73}$$

The MATLAB script shall be capable of supporting randomly generated binary antipodal data as well as white noise with a presettable value of N_0.

The MATLAB script shall consider the transmission of $W-1$ tail bits at the beginning of each data burst and of $W-1$ tail bits at the end of each data burst. Each tailbit shall be equal to -1. The MATLAB script shall be capable of plotting

- the unnoisy channel output vector $\boldsymbol{e}_{\mathrm{u}}$, which consists of the samples

$$e_{\mathrm{u},n} = \sum_{w=1}^{W} h_w d_{n-w+1}, \quad e_{\mathrm{u},n} \in \mathbb{R}, \tag{7.74}$$

- the noise vector $\boldsymbol{n}$, consisting of the noise samples $n_n \in \mathbb{R}$, and the
- received vector $\boldsymbol{e}$ which consists of the samples

$$e_n = \sum_{w=1}^{W} h_w d_{n-w+1} + n_n, \quad e_n \in \mathbb{R}. \tag{7.75}$$

In addition, the MATLAB script shall support the visualization of the channel trellis diagram, i.e the trellis diagram which represents the discrete-time transmission channel as a finite state machine in the form of a Mealy model with 2^{W-1} distinct states, including the highlighting of the path which is associated with the data vector

$$\left(d_1, d_2 \cdots d_N, \underbrace{-1, -1 \cdots -1}_{W-1 \text{ tailbits}}\right)^{\mathrm{T}}, \tag{7.76}$$

excluding the last $W-1$ samples, that are associated only with the transmission of the terminal $W-1$ tailbits. Clearly, the initial state of this channel trellis diagram is the state

$$\boldsymbol{\sigma}_0 = \left(\underbrace{-1, -1 \cdots -1}_{W-1 \text{ tailbits}}\right)^{\mathrm{T}} \tag{7.77}$$

with the state number

$$s_0 = \sum_{w=0}^{W-2} \frac{-1+1}{2} 2^w = \sum_{w=0}^{W-2} 0 = 0 \tag{7.78}$$

and the terminal state is the state

$$\boldsymbol{\sigma}_{N+W-1} = \left(\underbrace{-1, -1 \cdots -1}_{W-1 \text{ tailbits}}\right)^{\mathrm{T}}. \tag{7.79}$$

with the state number

$$s_{N+W-1} = \sum_{w=0}^{W-2} \frac{-1+1}{2} 2^w = \sum_{w=0}^{W-2} 0 = 0\,. \tag{7.80}$$

Furthermore, the MATLAB script shall support both the SOVA using the Forney metric and the SOVA using the Ungerboeck metric, simultaneously.

In addition, the SOVA implementations shall be capable of generating LLRs using the SIMPLE RULE, the HUBER RULE and the BATTAIL RULE, simultaneously, for both the SOVA using the Forney metric and the SOVA using the Ungerboeck metric.

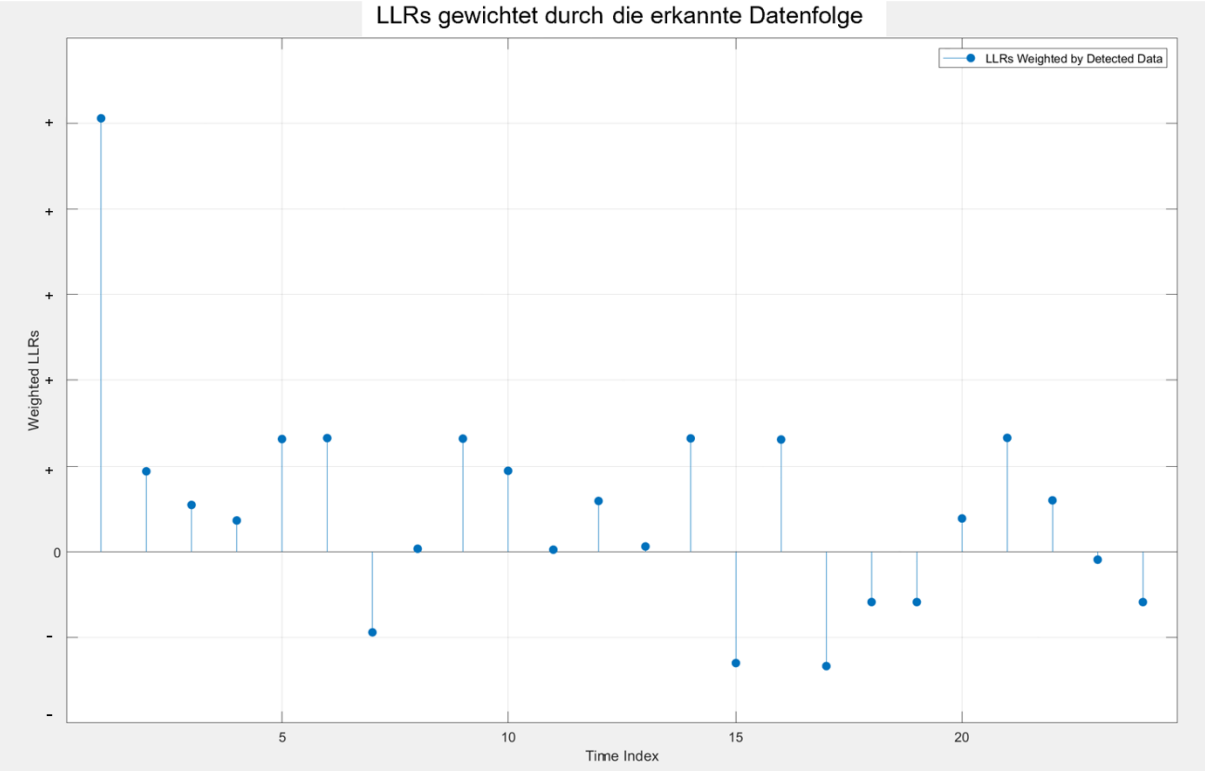
Also, the MATLAB script shall support the visualization of the trellis diagram at the detector including the highlighting of the path which is associated with the detected data.

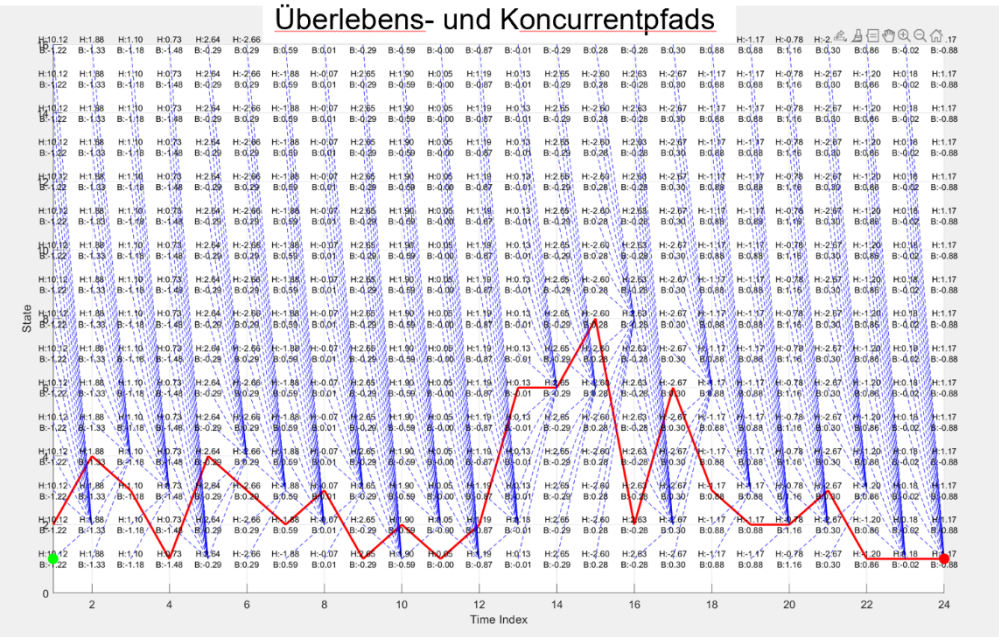
Furthermore, the MATLAB scipt shall be capable of illustrating the LLR updating procedures by the HUBER RULE and the BATTAIL RULE, by illustrating appropriate plots the survivor and the concurrent paths which terminate in the final state s_{N+W-1} labelling each of the states of the the survivor and the concurrent paths with the appropriate LLRs.

In addition, the MATLAB script shall be capable of plotting the detected data sequence in comparison with the transmitted data sequence, highlighting the locations where detection errors occur.

Finally, the MATLAB script shall be capable of plotting the generated LLRs at the SOVA outputs, weighted by the detected data.

LLR gewichtet durch die Erkannte Datenfolge: Dieses Diagramm zeigt die gewichtete LLR basierend auf den erkannten Daten. Wenn die Wahrscheinlichkeit groß ist, können wir sagen, dass unsere Entscheidung sicherer ist, als wenn sie nahe bei Null liegt.





Überlebens- und Konkurrentpfads: Diese beide Pfade representieren Überlebens- und Konkurrentpfads.

1.4.8. Aufgabe 8 :

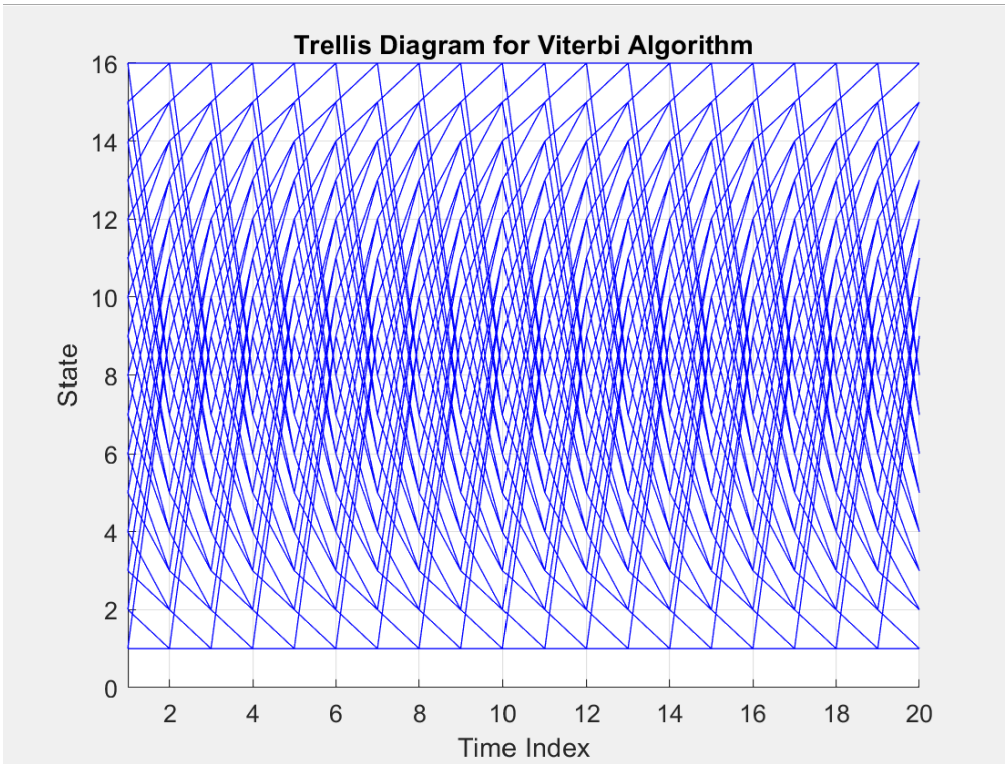
Plot the channel trellis diagram and the trellis diagram at the detector!

In the trellis diagram at the detector, plot those transitions which are associated with data symbols that were correctly detected in green and those transitions which are associated with data symbols that were erroneously detected in red.

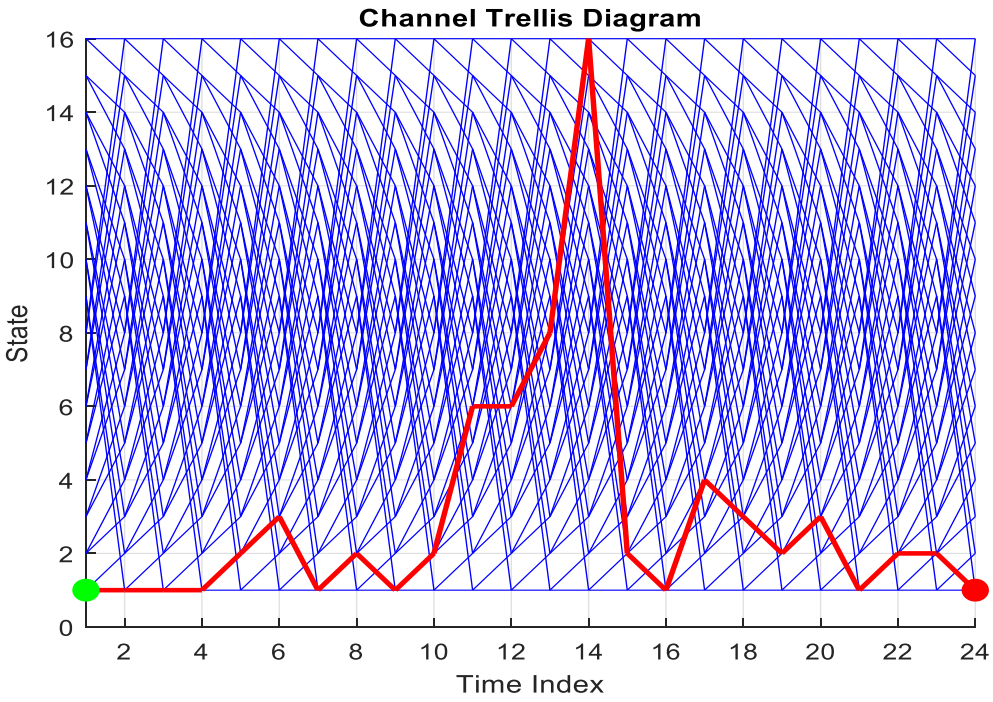
In dieser Frage wird das **Trellis-Diagramm des Kanals und des Detektors** gefordert, wobei die Übergänge entsprechend den detektierten Daten in diesem Diagramm ebenfalls dargestellt werden sollen.

Das resultierende Trellis-Diagramm ist in der folgenden Abbildung dargestellt.

Kanal Trellis Diagramm: Das Ziel eines Kanal-Trellis-Diagramms ist es, den optimalen Pfad zu finden, der durch unseren Algorithmus erkannt wird.



Trellis Diagramm zeigt uns die Zustände eines Systems über die Zeit und die mögliche Übergänge zwischen ihnen.Dann Viterbi Algorithmus verwendet, um den optimalen Pfad durch minimierung der Pfadmetriken zu finden.

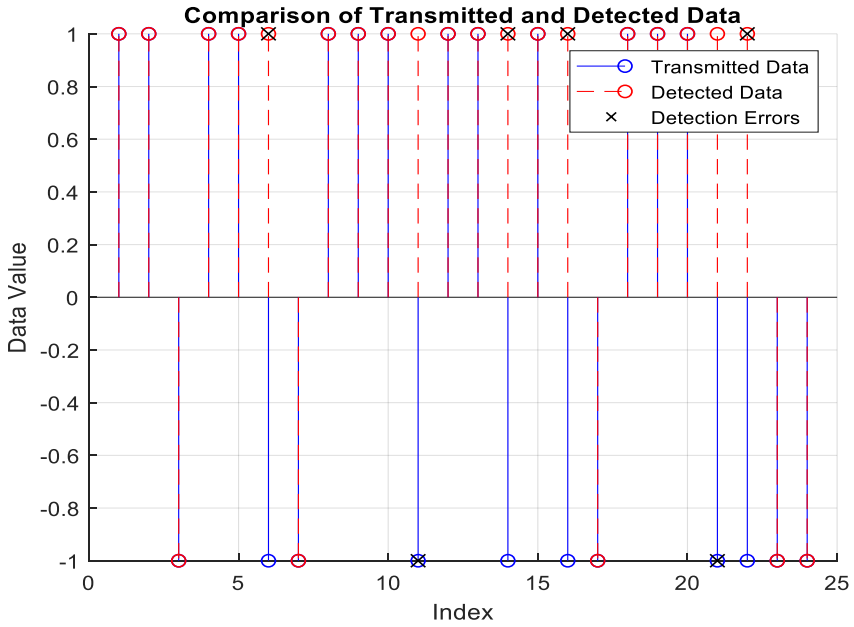


1.4.9. Aufgabe 9:

- Plot the transmitted binary antipodal data sequence alongside the detected binary antipodal data sequence and highlight where the SOVA detector made detection errors!

Vergleich von gesendeten und detektierten Daten: Die folgende Abbildung zeigt den Vergleichdieser beiden Datensätze. Dieses Diagramm vergleicht die gesendeten Daten

Erklärung:



1. **Gesendete Daten (blaue Linie mit Kreisen):** Die Daten, die als Eingabe in das System oder die Kommunikationskanäle gesendet wurden.
2. **Erkannte Daten (rote Linie mit Kreisen):** Die Daten, die vom Empfänger oder System nach der Verarbeitung erkannt wurden.
3. **Erkennungsfehler (schwarze Kreuze):** Die Abweichungen zwischen den gesendeten und den erkannten Daten, die auf Fehler im Erkennungssystem hinweisen.

Analyse:

- Wenn die **gesendeten und erkannten Daten** vollständig übereinstimmen, gibt es keine Erkennungsfehler (schwarze Kreuze).
- Im Diagramm sieht man, dass nur wenige Punkte Fehler aufweisen, was auf eine relativ gute Leistung des Erkennungssystems hindeutet.
- Abweichungen in den Werten (z. B. zwischen 1 und -1) könnten durch Rauschen oder Mängel im Erkennungsalgorithmus verursacht werden.

1.4.10. Aufgabe 10:

Plot the unnoisy received vector $\boldsymbol{e}_{\mathrm{u}}$, i.e. the unnoisy vector at the channel output, with the coordinates

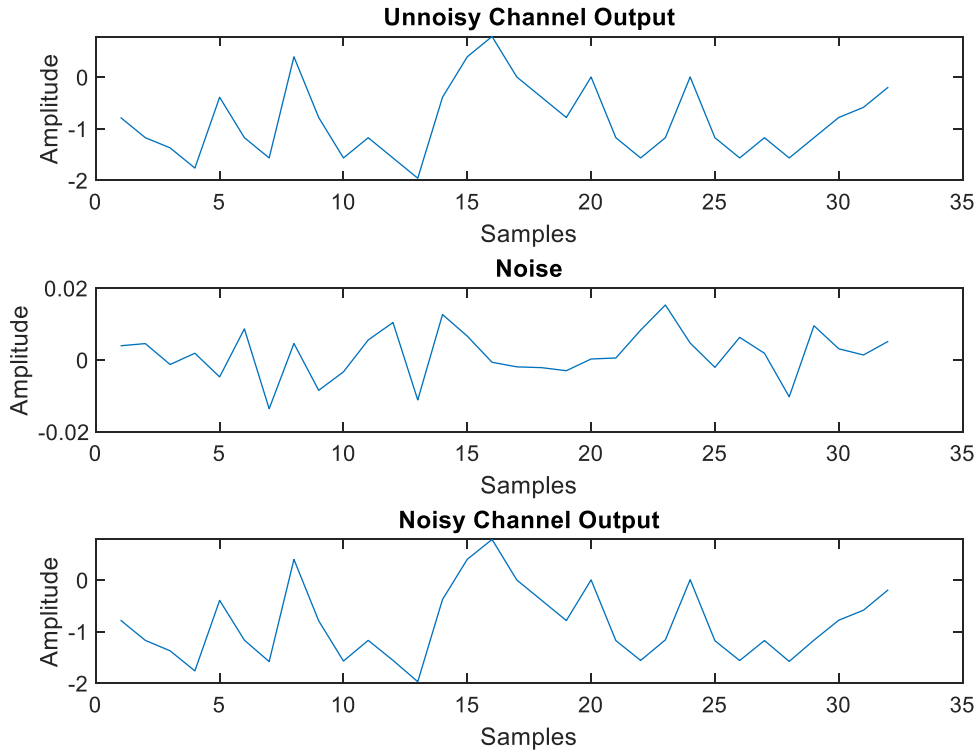
$$c_{\mathrm{u},n} = \sum_{w=1}^{W} h_w d_{k-w+1}, \quad c_{\mathrm{u},n} \in \mathbb{R}, \tag{7.81}$$

the white noise vector $\boldsymbol{n}$ at the receiver input with the coordinates $n_n \in \mathbb{R}$ and the received vector $\boldsymbol{e}$ with the coordinates

$$c_n = c_{\mathrm{u},n} + n_u, \quad c_u \in \mathbb{R}, \tag{7.82}$$

at the receiver input!

In dieser Frage werden erneut das **verrauschte Signal**, das **rauschfreie Signal** und das **Kanalrauschen** dargestellt, ähnlich wie in Frage 7. Der entsprechende Code für diesen Abschnitt wurde separat in der Datei **main10.m** implementiert.
Die resultierenden Diagramme zeigen die genannten Signale und deren Vergleich.



1.4.11. Aufgabe 11:

Plot the LLR values!

Berechnung der LLR-Werte mit den drei Regeln [6]

Zur Berechnung der **Likelihood Ratio (LLR)** in jedem Zustand werden drei verschiedene Regeln verwendet. Diese berechnen die Werte basierend auf rekursiven Beziehungen entlang des Überlebenspfads (survivor path). Die Regeln sind wie folgt definiert:

2 Die einfache Regel (SIMPLE RULE):
Diese Regel berechnet den LLR-Wert für jeden Zustand im Zeitpunkt nn wie folgt:

$$L_{\mathrm{s},n} = \Delta_n^{(\mathrm{s})}, \quad n = 1,2\cdots N\,,$$

Dabei ist $\Delta_n^{(s)}$ die Differenz oder ein spezifischer Wert, der mit dem Zustand **s** im Zeitpunkt **n** assoziiert ist.

3 Die Huber-Regel (HUBER RULE):
Bei dieser Regel werden die LLR-Werte rekursiv aktualisiert und basieren auf verschiedenen Bedingungen:

Diese Beziehung zeigt, dass die LLR-Werte entlang des Überlebenspfads abhängig von der Übereinstimmung der Daten mit dem Kandidatenpfad und den vorherigen Zuständen aktualisiert werden.

$$L_{\mathrm{h},\kappa}^{(\mathrm{s}),\mathrm{new}} = \begin{cases} \Delta_n^{(\mathrm{s})} & \kappa = n, \\ L_{\mathrm{h},\kappa}^{(\mathrm{s})} & \kappa < n,\ \underline{d}_\kappa^{(\mathrm{s})} = \underline{d}_\kappa^{(\mathrm{c})}, \\ \min\left\{L_{\mathrm{h},\kappa}^{(\mathrm{s})}, \Delta_n^{(\mathrm{s})}\right\} & \kappa < n,\ \underline{d}_\kappa^{(\mathrm{s})} \neq \underline{d}_\kappa^{(\mathrm{c})}, \end{cases}$$

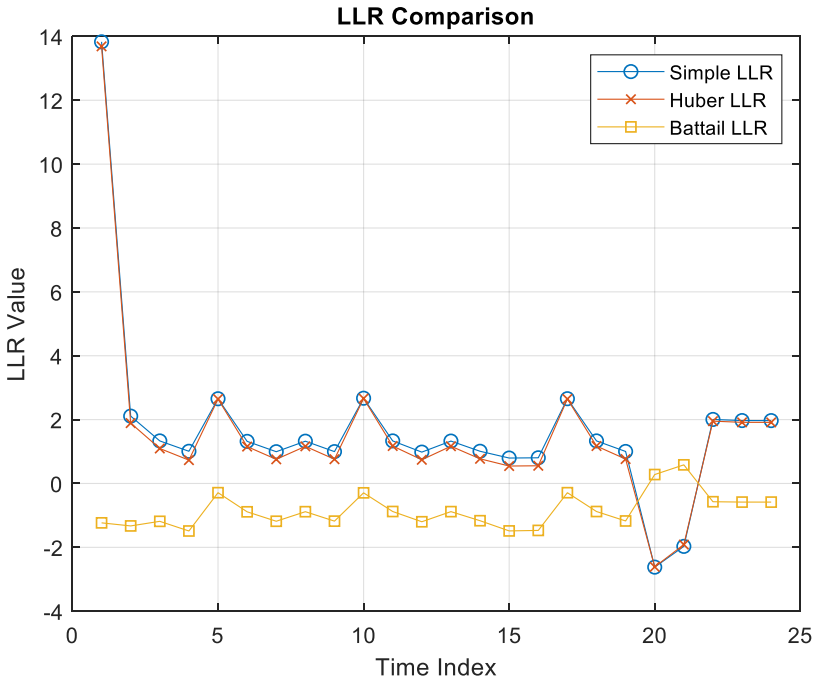
4 Die Battail-Regel (BATTAIL RULE):
Diese Regel aktualisiert die LLR-Werte ebenfalls rekursiv und verwendet eine komplexere Kombination:

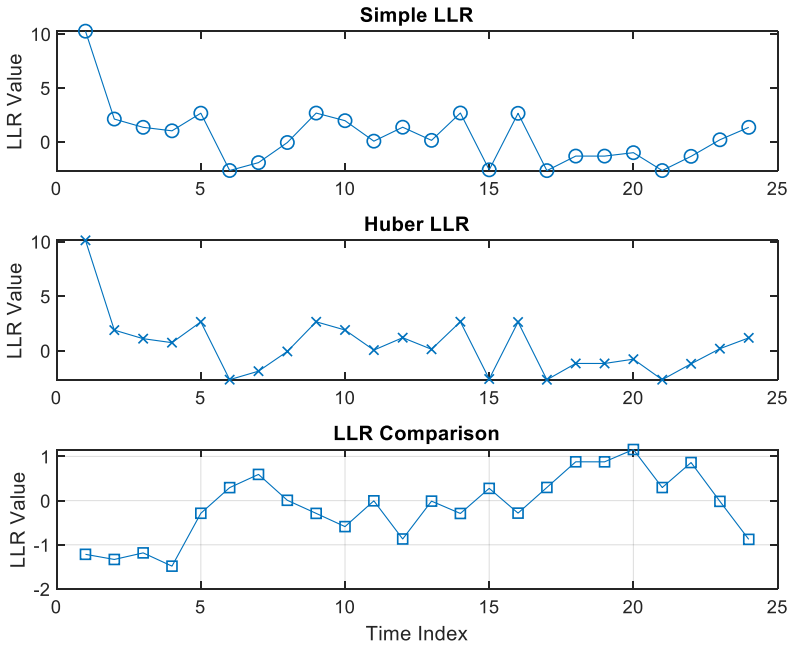
$$L_{\mathrm{b},\kappa}^{(\mathrm{s}),\mathrm{new}} = \begin{cases} \Delta_n^{(\mathrm{s})} & \kappa = n, \\ \min\left\{L_{\mathrm{b},\kappa}^{(\mathrm{s})}, \left[\Delta_n^{(\mathrm{s})} + L_{\mathrm{b},\kappa}^{(\mathrm{c})}\right]\right\} & \kappa < n,\ \underline{d}_\kappa^{(\mathrm{s})} = \underline{d}_\kappa^{(\mathrm{c})}, \\ \min\left\{L_{\mathrm{b},\kappa}^{(\mathrm{s})}, \Delta_n^{(\mathrm{s})}\right\} & \kappa < n,\ \underline{d}_\kappa^{(\mathrm{s})} \neq \underline{d}_\kappa^{(\mathrm{c})}. \end{cases}$$

Hierbei werden neben $\Delta_n^{(s)}$ auch die Kandidatenwerte $L_{b,k}^{(c)}$ in die Berechnungen einbezogen.

[6] Prof.Peter Jung : Advanced Mobile Communications

- Jede Regel wurde für spezifische Situationen entwickelt, und ihre Leistung hängt von den Eigenschaften des Algorithmus und der Überlebenspfade ab.
- Die Werte $\Delta_n^{(s)}$, $L_{h,k}^{(s)}$ oder $L_{b,k}^{(c)}$ können basierend auf der gewählten Metrik (Forney oder Ungerboeck), die durch den Parameter **METRIC** festgelegt ist, berechnet werden.





1. **SIMPLE RULE (blaue Kreise):**

- Diese Regel ist die einfachste Methode zur Berechnung des LLR und hängt nur vom aktuellen Wert $\Delta_n^{(s)}$ ab.
- Das Diagramm zeigt, dass diese Methode schnelle Schwankungen aufweist, da sie direkt von den Daten jedes Zeitpunkts beeinflusst wird.
- Aufgrund des Fehlens von Informationen aus vorherigen Zuständen ist diese Methode anfälliger für Rauschen und zeigt stärkere Schwankungen.

2. **HUBER RULE (rote Kreuze):**

Diese Regel verwendet Informationen aus vorherigen Zuständen $L_{h,k}^{(s)}$und vergleicht $d_k^{(s)}$mit $d_k^{(c)}$.

- Das Diagramm zeigt, dass diese Methode im Vergleich zu SIMPLE LLR glattere Verläufe aufweist und weniger Schwankungen zeigt.
- Dieses Verhalten ergibt sich aus der "Minimal"-Eigenschaft der Regel, die unnötige Schwankungen begrenzt.

3. **BATTAIL RULE (gelbe Quadrate):**

- Diese Regel ist die komplexeste und kombiniert Informationen aus vorherigen Zuständen sowie aktuelle Daten.
- Das Diagramm zeigt, dass diese Methode die LLR-Werte in einem engeren Bereich hält und ein stabileres Verhalten sowie weniger Rauschen aufweist.
- Im Vergleich zu den anderen beiden Regeln aktualisiert BATTAIL die LLR-Werte kontrollierter und gleichmäßiger.

1.4.12. Aufgabe 12:

Implement a section in your MATLAB script for the illustration of the LLR update procedure!

The section of your MATLAB script should set out from the final state

$$\mathfrak{s}_{N+W-1} = \left(\underbrace{-1,-1\cdots-1}_{W-1 \text{ tailbits}} \right)^{\mathrm{T}} \tag{7.83}$$

with the state number

$$s_{N+W-1} = \sum_{w=0}^{W-2} \frac{-1+1}{2} 2^w = \sum_{w=0}^{W-2} 0 = 0 \,. \tag{7.84}$$

The section of your MATLAB script should be capable of plotting both the survivor path which terminates in $\mathfrak{s}_{N+W-1}$ of (7.83) in green color and the concurrent path which terminates in $\mathfrak{s}_{N+W-1}$ of (7.83) in red color.

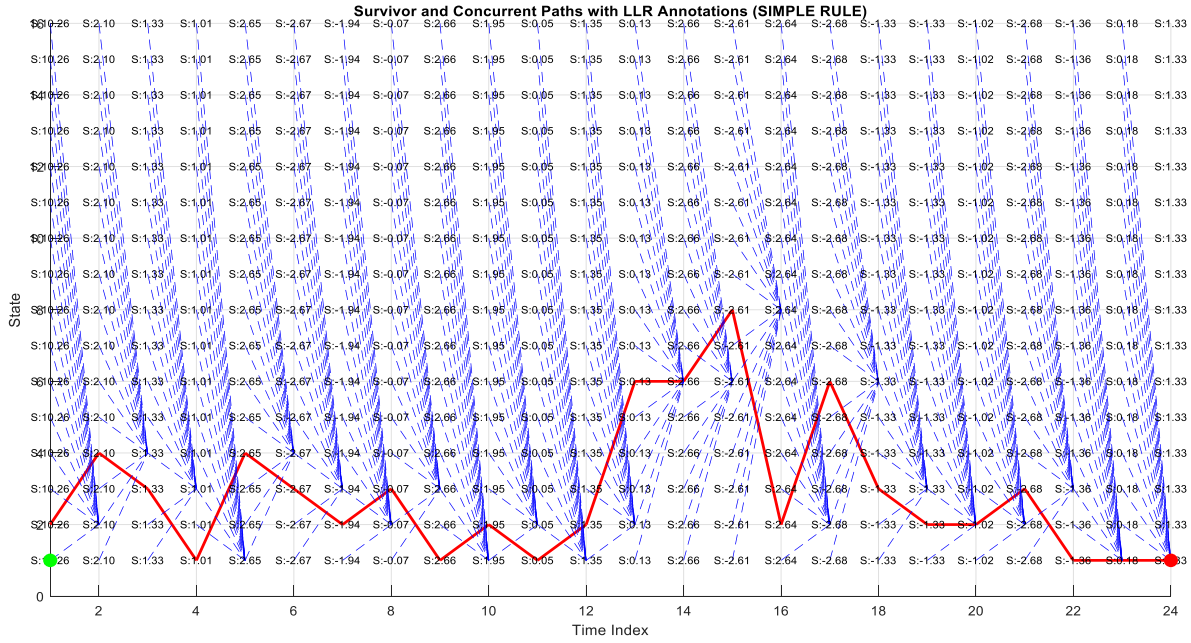
Since the survivor path and the concurrent path emerge from a common state $\mathfrak{s}_n$ in the path with $n < N+W-1$, the plots should commence at this common state $\mathfrak{s}_n$ and show the complete survivor path and the complete concurrent path until these merge at the final state $\mathfrak{s}_{N+W-1}$.

There should be separate plots for the SIMPLE RULE, the HUBER RULE and the BATTAIL RULE.

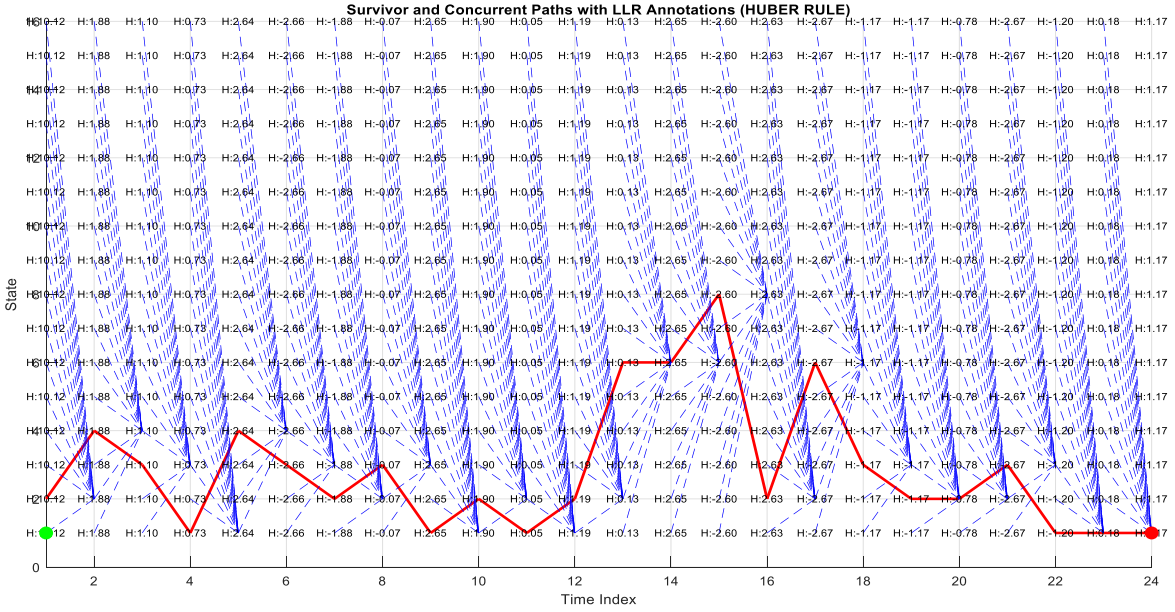
In each plot, the states of the survivor path and the concurrent path should be labelled with the corresponding LLR values which are assoviated with the transitions terminating in those states.

Ziel der Aufgabenstellung

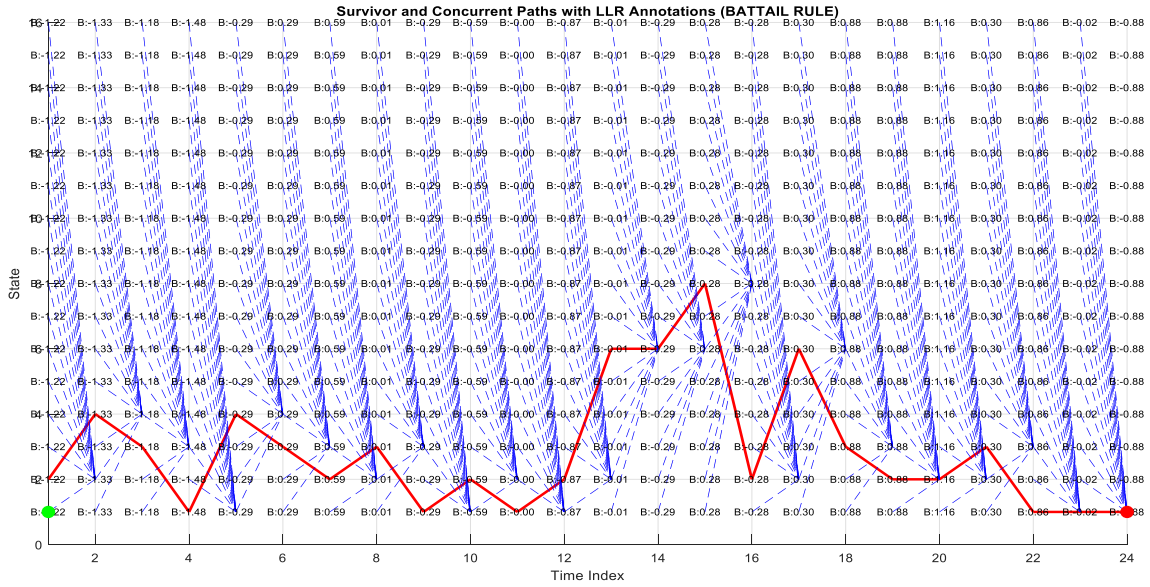
- Präzise Implementierung des Prozesses der LLR-Aktualisierung und Simulation der Survivor- und Concurrent-Pfade.
- Untersuchung und Vergleich der Pfade unter Verwendung der drei verschiedenen LLR-Regeln.
- Bereitstellung einer geeigneten grafischen Darstellung mit entsprechender Beschriftung der LLR-Werte.



Survivor and Concurrent Paths with LLR Annotations(SIMPLE RULE)



Survivor and Concurrent Paths with LLR Annotations(HUBER RULE)

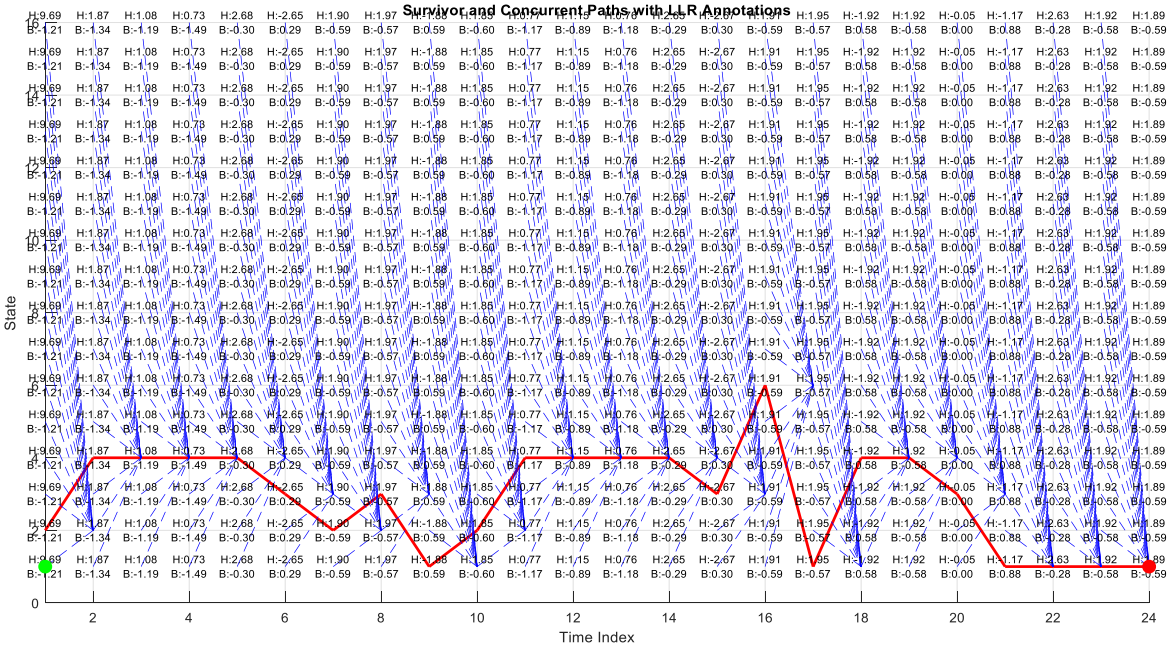


Survivor and Concurrent Paths with LLR Annotations(BATTAIL RULE)

1.4.13. Aufgabe 13:

Illustrate the LLR update procedure, considering the survivor path and the concurrent path which merge at the final state s_{N+W-1} of (7.83)!

Die Frage zielt darauf ab, den Aktualisierungsprozess der Log-Likelihood-Ratios (LLR) während der Ausführung des Viterbi-Algorithmus im Rahmen der Decodierung eines Signals zu untersuchen. Der Viterbi-Algorithmus wird verwendet, um den besten Pfad (oder die beste Sequenz) in einem Graphen mit verschiedenen Zuständen zu finden. Der Schwerpunkt dieser Frage liegt auf der Aktualisierung der LLR-Werte und dem Vergleich zwischen den verschiedenen Pfaden (Survivor Path und Concurrent Paths).



Survivor and Concurrent Path with LLR Annotations

1.4.14. Aufgabe 15:

Plot the single-hop section plots of the detected trellis diagram for the following settings: The power spectral density of the noise is

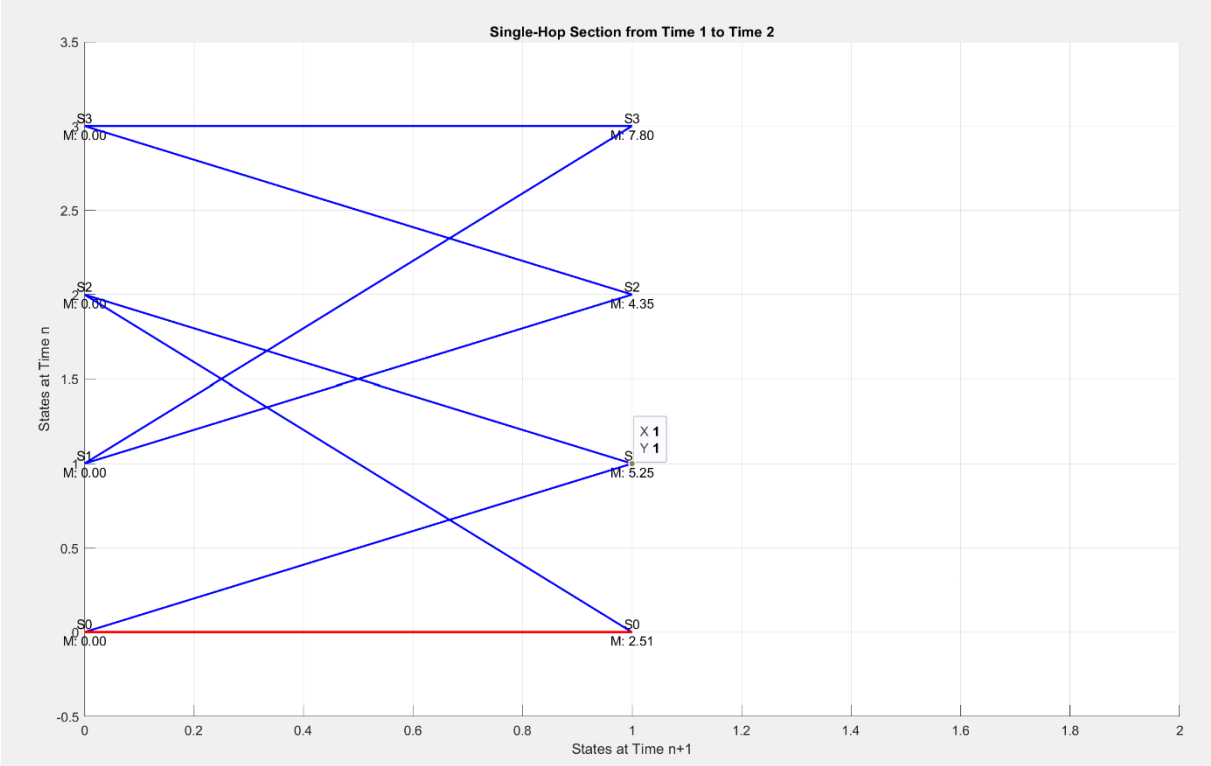
$$N_0 = 1, \tag{7.85}$$

The number of randomly chosen binary data symbols is

$$N = 3. \tag{7.86}$$

The channel impulse response shall be

$$\boldsymbol{h} = \left(\frac{1}{2}, \frac{1}{\sqrt{2}}, \frac{1}{2}\right)^{\mathrm{T}}. \tag{7.87}$$



1.5. Referenzen :

- Prof.Peter Jung : Advanced Mobile Communications
- Forney, G. D. Jr., "The Viterbi Algorithm," *Proceedings of the IEEE*, 1973
- G. Ungerboeck, "Channel coding with multilevel/phase signals," in IEEE Transactions on Information Theory, vol. 28, no. 1, pp. 55-67, January 1982, doi: 10.1109/TIT.1982.1056454.